

Quantium Virtual Internship

Retail Strategy & Analytics — Task 1

Data preparation, customer analytics, initial findings, and reproducible Python code

Analysis period: 1 July 2018–30 June 2019

Executive recommendation

Prioritise **Mainstream young singles/couples** as the clearest growth segment: they are one of the largest sales pools, pay a statistically higher price per pack than comparable Budget/Premium shoppers, and show distinct preferences for TYRRELLS and TWISTIES. Use prominent displays and targeted promotions around those preferences. In parallel, retain the high-volume family base—especially Budget older families—with multipack/value mechanics rather than blanket discounting.

Commercial actions

1. Target Mainstream young singles/couples with brand-led displays and offers centred on their over-indexing brands and pack sizes.
2. Protect family volume using value bundles and larger-basket mechanics; these customers buy more units per shopper.
3. Avoid category-wide price cuts: the target segment already tolerates a higher unit price, so targeted activation should preserve margin.
4. Test the recommendation in selected stores and measure incremental units, revenue, margin, penetration, and repeat purchase against controls.

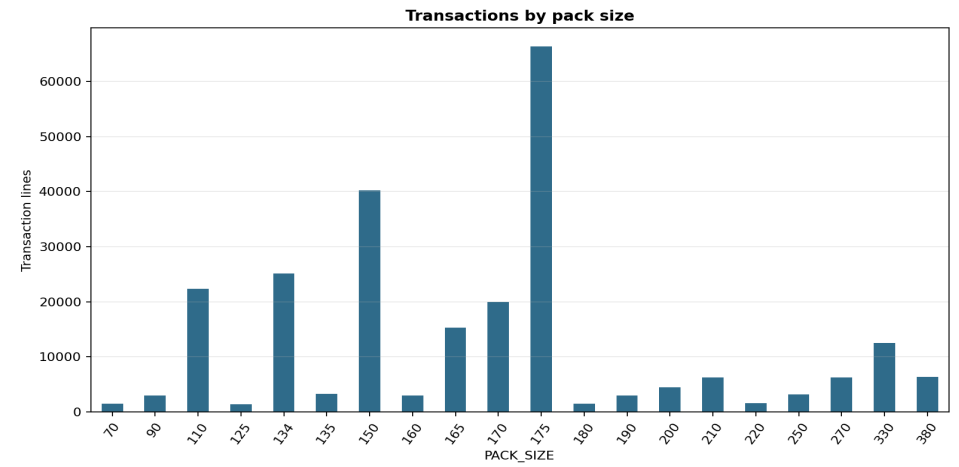
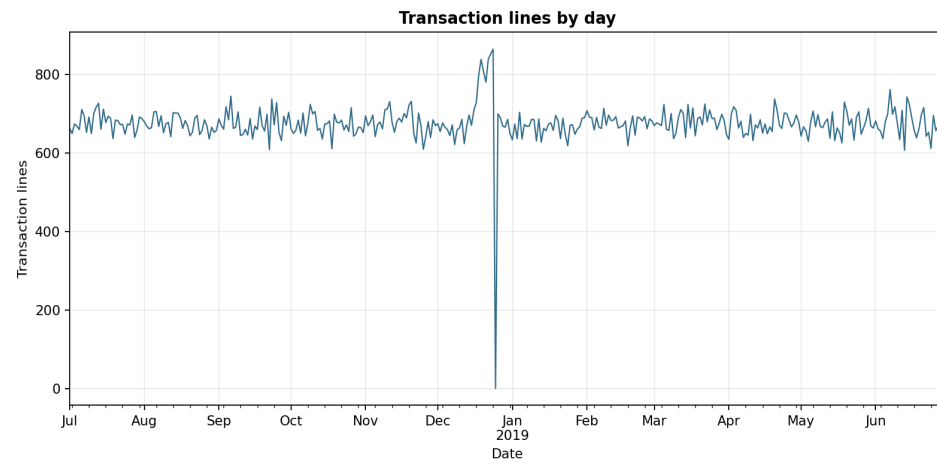
Important interpretation

Affinity is descriptive, not causal. It compares the target segment's unit mix with everyone else's mix. A controlled trial is needed before a national rollout.

Data preparation and validation

Check	Result
Raw transaction rows	264,836
Raw transaction null cells	0
Customer null cells	0
Duplicate customer IDs	0
Salsa rows excluded	18,094
Bulk-buyer card(s) excluded	226000
Bulk transaction rows	2
Clean chip rows	246,740
Unmatched joined transactions	0
Observed date range	2018-07-01 to 2019-06-30
Zero-transaction dates	2018-12-25
Pack-size range	70g to 380g

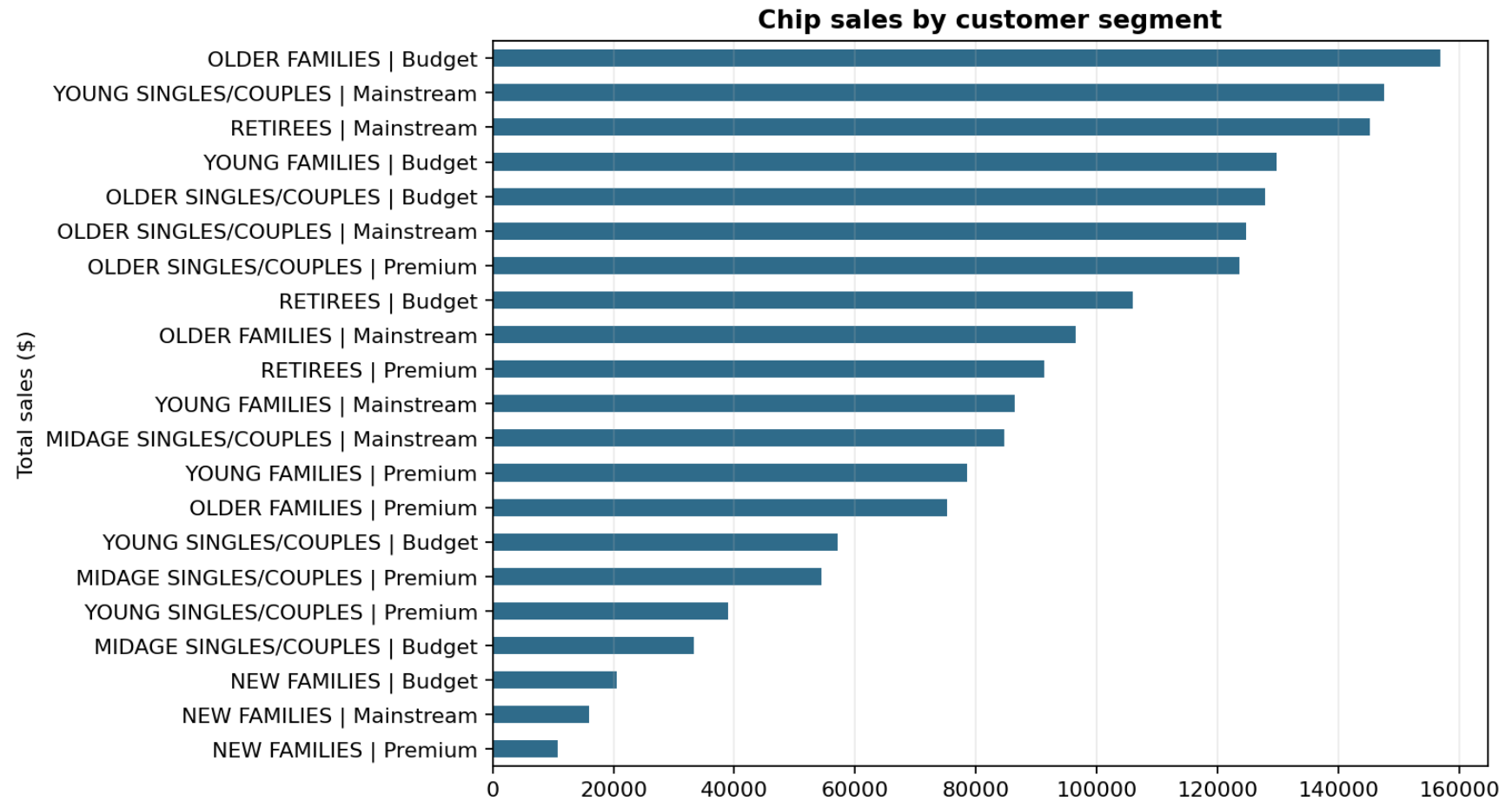
The 200-unit purchases belonged to one card with only two transactions and were removed as non-retail behaviour. Christmas Day is the only zero-transaction day and is operationally plausible. All customer joins matched.



Who drives chip sales?

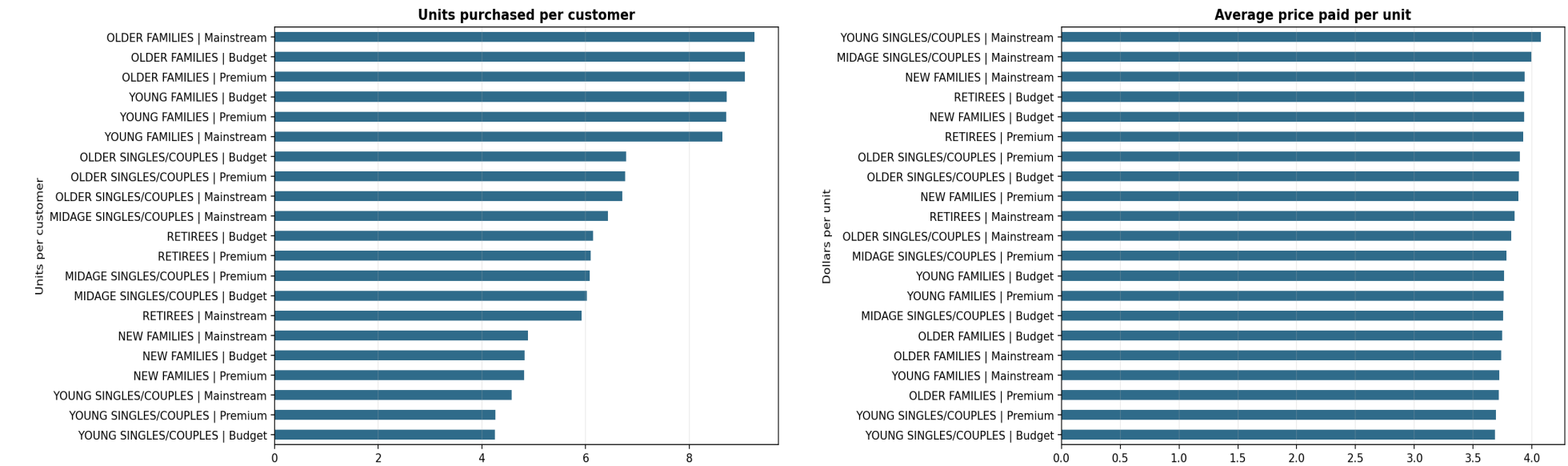
The leading sales pools combine customer count, purchase frequency/volume, and price. The top five are shown below.

Segment	Total Sales	Sales Share	Customers
OLDER FAMILIES Budget	\$156,863.75	8.7%	4611
YOUNG SINGLES/COUPLES Mainstream	\$147,582.20	8.2%	7917
RETIREEES Mainstream	\$145,168.95	8.0%	6358
YOUNG FAMILIES Budget	\$129,717.95	7.2%	3953
OLDER SINGLES/COUPLES Budget	\$127,833.60	7.1%	4849



Drivers of spend: volume and price

The highest unit-per-customer segments are led by OLDER FAMILIES | Mainstream (9.26 units/customer). The highest average unit price is YOUNG SINGLES/COUPLES | Mainstream (\$4.07).



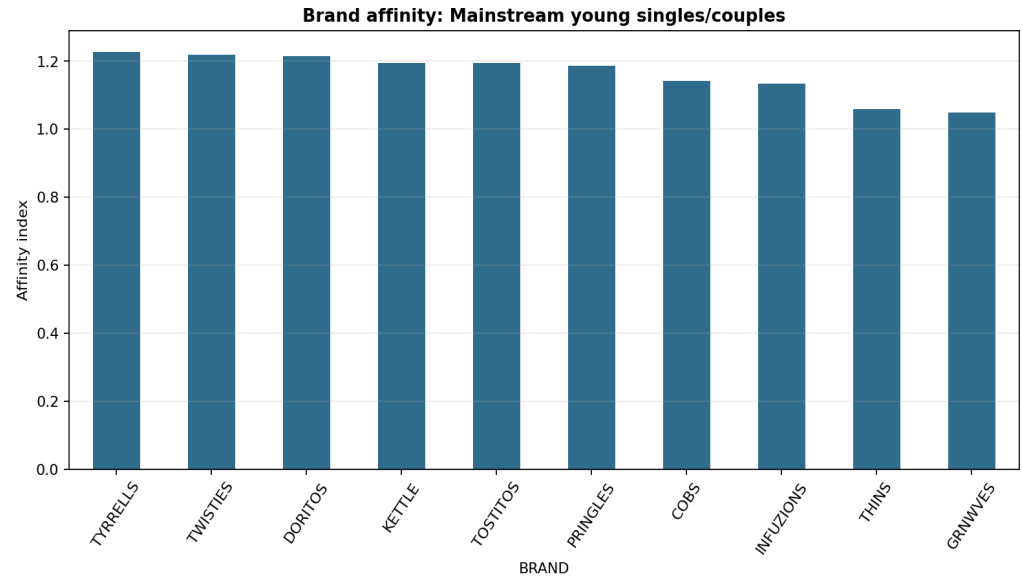
Welch one-sided t-test for mid-age and young singles/couples: Mainstream mean = \$4.04, Budget/Premium mean = \$3.71; $t = 37.624$, $p = 3.48e-306$. The Mainstream unit price is significantly higher at the 5% level.

Deep dive: Mainstream young singles/couples

This segment contains 7,917 chip buyers and contributes \$147,582.20 in sales. An affinity above 1.00 means the item has a higher unit share in the target than among all other chip buyers.

Brand affinity

Brand	Target Share	Other Share	Affinity
TYRRELLS	3.16%	2.57%	1.23x
TWISTIES	4.62%	3.79%	1.22x
DORITOS	12.28%	10.11%	1.21x
KETTLE	19.80%	16.56%	1.20x
TOSTITOS	4.54%	3.80%	1.20x
PRINGLES	11.94%	10.06%	1.19x
COBS	4.46%	3.90%	1.14x
INFUZIONI	6.47%	5.71%	1.13x



Pack-size affinity

Pack Size	Target Share	Other Share	Affinity
270	3.18%	2.51%	1.27x
380	3.22%	2.56%	1.26x
330	6.13%	5.02%	1.22x
134	11.94%	10.06%	1.19x
110	10.63%	8.98%	1.18x
210	2.91%	2.51%	1.16x
135	1.48%	1.31%	1.13x
250	1.44%	1.28%	1.12x

The apparent 270g preference is not an independent pack-size effect: Twisties Cheese and Twisties Chicken are the only 270g products. Treat it as supporting evidence for Twisties affinity, not as proof that any 270g pack will perform better.

Method and limitations

Metrics: total chip sales; unique chip-buying customers; units/customer; weighted average dollars/unit; sales share; and target-versus-rest affinity by unit mix. Brand and pack size are derived from product names after known aliases are standardised.

Limitations

The data covers chip purchasers and does not include total grocery spend, margin, household demographics beyond the supplied segments, promotion exposure, store availability, or a non-buyer benchmark. Therefore the analysis identifies associations and commercial hypotheses, not incremental causal effects. Statistical testing uses transaction lines, which may understate dependence among repeated purchases by the same customer.

Suggested next analysis

Add promotion, margin, store, and total-basket data; assess penetration relative to the full customer base; and validate activation via matched test/control stores. Track incremental revenue and margin—not only affinity or raw sales.

Reproducibility

Inputs: QVI_transaction_data.xlsx, QVI_purchase_behaviour.csv. Clean output: QVI_data.csv. The complete executable source follows in the appendix.

Appendix: complete Python code

"""Quantium Task 1: data preparation and customer analytics.

Reads the original files from the user's Downloads folder, cleans and joins them, creates customer-segment metrics, runs an affinity analysis, and produces a PDF submission containing findings, charts, validation evidence, and this source code.
"""

```
from pathlib import Path
import re
import textwrap
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import ttest_ind
from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4, landscape
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer, PageBreak, Image, Table, TableStyle,
    Preformatted, KeepTogether
)

warnings.filterwarnings("ignore", category=UserWarning, module="openpyxl")

ROOT = Path(__file__).resolve().parent
DOWNLOADS = Path.home() / "Downloads"
TX_PATH = DOWNLOADS / "QVI_transaction_data.xlsx"
CUSTOMER_PATH = DOWNLOADS / "QVI_purchase_behaviour.csv"
OUTPUT_DATA = ROOT / "QVI_data.csv"
OUTPUT_PDF = ROOT / "Quantium_Task1_Initial_Findings_and_Code.pdf"
CHART_DIR = ROOT / "quantium_charts"
CHART_DIR.mkdir(exist_ok=True)

def money(x):
    return f"${x:,.2f}"

def clean_brand(name):
    brand = str(name).split()[0].upper()
    corrections = {
        "RED": "RRD", "SNBTS": "SUNBITES", "INFZNS": "INFUZIONI",
        "WM": "WOOLWORTHS", "SMITH": "SMITHS", "DORITO": "DORITOS",
        "NCC": "NATURAL", "GRAIN": "GRNWWES",
    }
    return corrections.get(brand, brand)

def save_bar(series, title, ylabel, filename, horizontal=False):
    fig, ax = plt.subplots(figsize=(10, 5.5))
    if horizontal:
        series.sort_values().plot(kind="barh", ax=ax, color="#2F6B8A")
    else:
        series.plot(kind="bar", ax=ax, color="#2F6B8A")
        ax.tick_params(axis="x", rotation=55)
    ax.set_title(title, weight="bold")
    ax.set_ylabel(ylabel)
    ax.grid(axis="y" if not horizontal else "x", alpha=.25)
    fig.tight_layout()
    path = CHART_DIR / filename
    fig.savefig(path, dpi=160, bbox_inches="tight")
    plt.close(fig)
    return path

def df_table(df, widths=None, font=7):
    printable = df.copy()
    printable.columns = [str(c).replace("_", " ").title() for c in printable.columns]
    data = [list(printable.columns)] + printable.astype(str).values.tolist()
    table = Table(data, colWidths=widths, repeatRows=1, hAlign="LEFT")
    table.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#17365D")),
    ]))
```

```

        ("TEXTCOLOR", (0, 0), (-1, 0), colors.white),
        ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
        ("FONTNAME", (0, 1), (-1, -1), "Helvetica"),
        ("FONTSIZE", (0, 0), (-1, -1), font),
        ("GRID", (0, 0), (-1, -1), .25, colors.HexColor("#B8C2CC")),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, colors.HexColor("#F2F5F7")]),
        ("VALIGN", (0, 0), (-1, -1), "TOP"),
        ("LEFTPADDING", (0, 0), (-1, -1), 4),
        ("RIGHTPADDING", (0, 0), (-1, -1), 4),
    )))
return table

def page_number(canvas, doc):
    canvas.saveState()
    canvas.setFont("Helvetica", 8)
    canvas.setFillColor(colors.grey)
    canvas.drawRightString(285 * mm, 8 * mm, f"Page {doc.page}")
    canvas.restoreState()

def main():
    # Load and profile raw data.
    tx_raw = pd.read_excel(TX_PATH)
    customers = pd.read_csv(CUSTOMER_PATH)
    raw_rows = len(tx_raw)
    raw_nulls = int(tx_raw.isna().sum().sum())
    customer_nulls = int(customers.isna().sum().sum())
    duplicate_customer_ids = int(customers["LYLTY_CARD_NBR"].duplicated().sum())

    tx = tx_raw.copy()
    # Excel serial dates may arrive as integers or already-parsed timestamps.
    if pd.api.types.is_numeric_dtype(tx["DATE"]):
        tx["DATE"] = pd.to_datetime(tx["DATE"], unit="D", origin="1899-12-30")
    else:
        tx["DATE"] = pd.to_datetime(tx["DATE"])

    # Exclude salsa because it is outside the chips category.
    salsa_mask = tx["PROD_NAME"].str.contains("salsa", case=False, na=False)
    salsa_rows = int(salsa_mask.sum())
    tx = tx.loc[~salsa_mask].copy()

    # Investigate and remove the non-retail bulk buyer(s), rather than winsorising.
    outlier_rows = tx.loc[tx["PROD_QTY"] >= 200].copy()
    outlier_customers = sorted(outlier_rows["LYLTY_CARD_NBR"].unique().tolist())
    tx = tx.loc[~tx["LYLTY_CARD_NBR"].isin(outlier_customers)].copy()

    # Feature engineering from product name.
    tx["PACK_SIZE"] = tx["PROD_NAME"].str.extract(r"(\d+)(?=g\b)", flags=re.I)[0].astype(int)
    tx["BRAND"] = tx["PROD_NAME"].map(clean_brand)
    tx["UNIT_PRICE"] = tx["TOT_SALES"] / tx["PROD_QTY"]

    # Date completeness: a zero on Christmas Day is expected (stores closed).
    full_dates = pd.date_range(tx["DATE"].min(), tx["DATE"].max(), freq="D")
    daily = tx.groupby("DATE").size().reindex(full_dates, fill_value=0)
    missing_dates = daily[daily.eq(0)].index.strftime("%Y-%m-%d").tolist()

    # Join validation and export the cleaned analysis dataset.
    data = tx.merge(customers, on="LYLTY_CARD_NBR", how="left", validate="many_to_one")
    unmatched = int(data[["LIFESTAGE", "PREMIUM_CUSTOMER"]].isna().any(axis=1).sum())
    data.to_csv(OUTPUT_DATA, index=False)

    keys = ["LIFESTAGE", "PREMIUM_CUSTOMER"]
    segment = data.groupby(keys).agg(
        total_sales=("TOT_SALES", "sum"),
        customers=("LYLTY_CARD_NBR", "nunique"),
        units=("PROD_QTY", "sum"),
    ).reset_index()
    segment["units_per_customer"] = segment["units"] / segment["customers"]
    # Weighted price per unit: total dollars divided by total packs.
    segment["avg_price_per_unit"] = segment["total_sales"] / segment["units"]
    segment["sales_share"] = segment["total_sales"] / segment["total_sales"].sum()
    segment["segment"] = segment["LIFESTAGE"] + " | " + segment["PREMIUM_CUSTOMER"]

    top_sales = segment.nlargest(5, "total_sales").copy()
    top_sales_display = top_sales[["segment", "total_sales", "sales_share", "customers"]].copy()
    top_sales_display["total_sales"] = top_sales_display["total_sales"].map(money)
    top_sales_display["sales_share"] = top_sales_display["sales_share"].map(lambda x: f"{x:.1%}")

    top_units = segment.nlargest(5, "units_per_customer")

```

```

top_price = segment.nlargest(5, "avg_price_per_unit")

# Price significance test requested in the template.
age_mask = data["LIFESTAGE"].isin(["MIDAGE SINGLES/COUPLES", "YOUNG SINGLES/COUPLES"])
mainstream_prices = data.loc[age_mask & data["PREMIUM_CUSTOMER"].eq("Mainstream"), "UNIT_PRICE"]
other_prices = data.loc[age_mask & ~data["PREMIUM_CUSTOMER"].eq("Mainstream"), "UNIT_PRICE"]
test = ttest_ind(mainstream_prices, other_prices, equal_var=False, alternative="greater")

# Affinity analysis for Mainstream young singles/couples versus all others.
target_mask = data["LIFESTAGE"].eq("YOUNG SINGLES/COUPLES") & data["PREMIUM_CUSTOMER"].eq("Mainstream")
target = data.loc[target_mask]
rest = data.loc[~target_mask]

def affinity(field):
    target_mix = target.groupby(field)["PROD_QTY"].sum() / target["PROD_QTY"].sum()
    rest_mix = rest.groupby(field)["PROD_QTY"].sum() / rest["PROD_QTY"].sum()
    result = pd.concat([target_mix.rename("target_share"), rest_mix.rename("other_share")], axis=1).fillna(0)
    result["affinity"] = result["target_share"] / result["other_share"].replace(0, np.nan)
    return result.sort_values("affinity", ascending=False).reset_index()

brand_affinity = affinity("BRAND")
pack_affinity = affinity("PACK_SIZE")

brand_display = brand_affinity.head(8).copy()
pack_display = pack_affinity.head(8).copy()
for d in (brand_display, pack_display):
    d["target_share"] = d["target_share"].map(lambda x: f"{x:.2%}")
    d["other_share"] = d["other_share"].map(lambda x: f"{x:.2%}")
    d["affinity"] = d["affinity"].map(lambda x: f"{x:.2f}x")

# Charts.
sales_chart = save_bar(
    segment.set_index("segment")["total_sales"].sort_values(ascending=False),
    "Chip sales by customer segment", "Total sales ($)", "segment_sales.png", horizontal=True)
units_chart = save_bar(
    segment.set_index("segment")["units_per_customer"].sort_values(ascending=False),
    "Units purchased per customer", "Units per customer", "units_per_customer.png", horizontal=True)
price_chart = save_bar(
    segment.set_index("segment")["avg_price_per_unit"].sort_values(ascending=False),
    "Average price paid per unit", "Dollars per unit", "price_per_unit.png", horizontal=True)
pack_chart = save_bar(
    data.groupby("PACK_SIZE").size(), "Transactions by pack size", "Transaction lines", "pack_sizes.png")
affinity_chart = save_bar(
    brand_affinity.head(10).set_index("BRAND")["affinity"],
    "Brand affinity: Mainstream young singles/couples", "Affinity index", "brand_affinity.png")
fig, ax = plt.subplots(figsize=(10, 4.8))
daily.plot(ax=ax, color="#2F6B8A", linewidth=1)
ax.set_title("Transaction lines by day", weight="bold")
ax.set_xlabel("Date")
ax.set_ylabel("Transaction lines")
ax.grid(alpha=.25)
fig.tight_layout()
daily_chart = CHART_DIR / "daily_transactions.png"
fig.savefig(daily_chart, dpi=160, bbox_inches="tight")
plt.close(fig)

# Build a readable findings-and-code PDF in landscape orientation.
styles = getSampleStyleSheet()
styles.add(ParagraphStyle(name="ReportTitle", parent=styles["Title"], fontSize=22,
    leading=26, textColor=colors.HexColor("#17365D"), alignment=TA_CENTER))
styles.add(ParagraphStyle(name="H1x", parent=styles["Heading1"], fontSize=16,
    leading=19, textColor=colors.HexColor("#17365D"), spaceAfter=8))
styles.add(ParagraphStyle(name="H2x", parent=styles["Heading2"], fontSize=12,
    leading=15, textColor=colors.HexColor("#2F6B8A"), spaceAfter=6))
styles.add(ParagraphStyle(name="Bodyx", parent=styles["BodyText"], fontSize=9,
    leading=13, spaceAfter=7))
styles.add(ParagraphStyle(name="Smallx", parent=styles["BodyText"], fontSize=7.5,
    leading=10, spaceAfter=4))

doc = SimpleDocTemplate(str(OUTPUT_PDF), pagesize=landscape(A4),
    rightMargin=14*mm, leftMargin=14*mm,
    topMargin=13*mm, bottomMargin=13*mm)

story = []
story += [Spacer(1, 30*mm), Paragraph("Quantium Virtual Internship", styles["ReportTitle"]),
    Paragraph("Retail Strategy & Analytics – Task 1", styles["ReportTitle"]),
    Spacer(1, 8*mm), Paragraph("Data preparation, customer analytics, initial findings, and reproducible
        Python code", styles["Bodyx"]),
    Spacer(1, 4*mm), Paragraph("Analysis period: 1 July 2018–30 June 2019", styles["Bodyx"]), PageBreak()]

story += [Paragraph("Executive recommendation", styles["H1x"]),

```

```

Paragraph(
  "Prioritise <b>Mainstream young singles/couples</b> as the clearest growth segment: they are one of
  the largest "
  "sales pools, pay a statistically higher price per pack than comparable Budget/Premium shoppers, and
  show "
  f"distinct preferences for {brand_affinity.iloc[0]['BRAND']}} and TWISTIES. "
  "Use prominent displays and targeted promotions around those preferences. In parallel, retain the
  high-volume "
  "family base—especially Budget older families—with multipack/value mechanics rather than blanket
  discounting.", styles["Bodyx"]),
Paragraph("Commercial actions", styles["H2x"]),
Paragraph("1. Target Mainstream young singles/couples with brand-led displays and offers centred on
  their over-indexing brands and pack sizes.<br/>"
  "2. Protect family volume using value bundles and larger-basket mechanics; these customers buy
  more units per shopper.<br/>"
  "3. Avoid category-wide price cuts: the target segment already tolerates a higher unit price,
  so targeted activation should preserve margin.<br/>"
  "4. Test the recommendation in selected stores and measure incremental units, revenue, margin,
  penetration, and repeat purchase against controls.", styles["Bodyx"]),
Paragraph("Important interpretation", styles["H2x"]),
Paragraph("Affinity is descriptive, not causal. It compares the target segment's unit mix with everyone
  else's mix. A controlled trial is needed before a national rollout.", styles["Bodyx"]))

story += [Paragraph("Data preparation and validation", styles["H1x"])]
quality = pd.DataFrame([
  ["Raw transaction rows", f"{raw_rows:}",], ["Raw transaction null cells", raw_nulls],
  ["Customer null cells", customer_nulls], ["Duplicate customer IDs", duplicate_customer_ids],
  ["Salsa rows excluded", f"{salsa_rows:}",],
  ["Bulk-buyer card(s) excluded", ", ".join(map(str, outlier_customers))],
  ["Bulk transaction rows", len(outlier_rows)], ["Clean chip rows", f"{len(tx):}",],
  ["Unmatched joined transactions", unmatched],
  ["Observed date range", f"{tx['DATE'].min().date()} to {tx['DATE'].max().date()}",],
  ["Zero-transaction dates", ", ".join(missing_dates)],
  ["Pack-size range", f"{tx['PACK_SIZE'].min()}g to {tx['PACK_SIZE'].max()}g",],
], columns=["Check", "Result"])
story += [df_table(quality, widths=[75*mm, 105*mm], font=8, Spacer(1, 4*mm),
  Paragraph("The 200-unit purchases belonged to one card with only two transactions and were removed as
  non-retail behaviour. Christmas Day is the only zero-transaction day and is operationally plausible.
  All customer joins matched.", styles["Bodyx"]),
  Table([[Image(str(daily_chart), width=125*mm, height=62*mm),
    Image(str(pack_chart), width=125*mm, height=62*mm)]],
    colWidths=[130*mm, 130*mm]), PageBreak())

story += [Paragraph("Who drives chip sales?", styles["H1x"]),
  Paragraph("The leading sales pools combine customer count, purchase frequency/volume, and price. The top
  five are shown below.", styles["Bodyx"]),
  df_table(top_sales_display, widths=[80*mm, 35*mm, 30*mm, 30*mm], font=8, Spacer(1, 4*mm),
  Image(str(sales_chart), width=230*mm, height=125*mm), PageBreak())

story += [Paragraph("Drivers of spend: volume and price", styles["H1x"]),
  Paragraph(f"The highest unit-per-customer segments are led by {top_units.iloc[0]['segment']}
  ({top_units.iloc[0]['units_per_customer']:.2f} units/customer). "
  f"The highest average unit price is {top_price.iloc[0]['segment']}
  ({money(top_price.iloc[0]['avg_price_per_unit'])}).", styles["Bodyx"]),
  Table([[Image(str(units_chart), width=125*mm, height=78*mm), Image(str(price_chart), width=125*mm,
    height=78*mm)]], colWidths=[130*mm, 130*mm]),
  Spacer(1, 4*mm),
  Paragraph(f"Wech one-sided t-test for mid-age and young singles/couples: Mainstream mean =
  {money(mainstream_prices.mean())}, Budget/Premium mean = {money(other_prices.mean())}; t =
  {test.statistic:.3f}, p = {test.pvalue:.3g}. The Mainstream unit price is significantly higher at
  the 5% level.", styles["Bodyx"]), PageBreak())

story += [Paragraph("Deep dive: Mainstream young singles/couples", styles["H1x"]),
  Paragraph("This segment contains {target['LYLTY_CARD_NBR'].nunique():} chip buyers and contributes
  {money(target['TOT_SALES'].sum())} in sales. An affinity above 1.00 means the item has a higher unit
  share in the target than among all other chip buyers.", styles["Bodyx"]),
  Paragraph("Brand affinity", styles["H2x"]),
  Table([[df_table(brand_display, widths=[32*mm, 28*mm, 28*mm, 25*mm], font=7),
    Image(str(affinity_chart), width=135*mm, height=78*mm)]], colWidths=[118*mm, 140*mm]),
  Spacer(1, 3*mm), Paragraph("Pack-size affinity", styles["H2x"]),
  df_table(pack_display, widths=[32*mm, 35*mm, 35*mm, 28*mm], font=7),
  Spacer(1, 3*mm),
  Paragraph("The apparent 270g preference is not an independent pack-size effect: Twisties Cheese and
  Twisties Chicken are the only 270g products. Treat it as supporting evidence for Twisties affinity,
  not as proof that any 270g pack will perform better.", styles["Bodyx"]), PageBreak())

story += [Paragraph("Method and limitations", styles["H1x"]),
  Paragraph("Metrics: total chip sales; unique chip-buying customers; units/customer; weighted average
  dollars/unit; sales share; and target-versus-rest affinity by unit mix. Brand and pack size are
  derived from product names after known aliases are standardised.", styles["Bodyx"]),

```

```

Paragraph("Limitations", styles["H2x"]),
Paragraph("The data covers chip purchasers and does not include total grocery spend, margin, household
demographics beyond the supplied segments, promotion exposure, store availability, or a non-buyer
benchmark. Therefore the analysis identifies associations and commercial hypotheses, not incremental
causal effects. Statistical testing uses transaction lines, which may understate dependence among
repeated purchases by the same customer.", styles["Bodyx"]),
Paragraph("Suggested next analysis", styles["H2x"]),
Paragraph("Add promotion, margin, store, and total-basket data; assess penetration relative to the full
customer base; and validate activation via matched test/control stores. Track incremental revenue
and margin-not only affinity or raw sales.", styles["Bodyx"]),
Paragraph("Reproducibility", styles["H2x"]),
Paragraph(f"Inputs: {TX_PATH.name}, {CUSTOMER_PATH.name}. Clean output: {OUTPUT_DATA.name}. The complete
executable source follows in the appendix.", styles["Bodyx"]), PageBreak()

story += [Paragraph("Appendix: complete Python code", styles["H1x"])]
source = Path(__file__).read_text(encoding="utf-8")
# Wrap long lines for a printable code appendix.
wrapped = []
for line in source.splitlines():
    if len(line) <= 118:
        wrapped.append(line)
    else:
        indent = len(line) - len(line.lstrip())
        wrapped.extend(textwrap.wrap(line, width=118, subsequent_indent=" " * (indent + 4),
                                   break_long_words=False, break_on_hyphens=False))
story.append(Preformatted("\n".join(wrapped), ParagraphStyle(
    name="Code", fontName="Courier", fontSize=5.1, leading=6.3,
    textColor=colors.HexColor("#1F2933"), leftIndent=0, rightIndent=0)))

doc.build(story, onFirstPage=page_number, onLaterPages=page_number)

print(f"Raw rows: {raw_rows:,}")
print(f"Clean chip rows: {len(data):,}")
print(f"Clean sales: {money(data['TOT_SALES'].sum())}")
print("Top sales segments:")
print(top_sales_display.to_string(index=False))
print(f"T-test p-value: {test.pvalue:.8g}")
print("Top brand affinities:")
print(brand_affinity.head(5).to_string(index=False))
print("Top pack affinities:")
print(pack_affinity.head(5).to_string(index=False))
print(f"Wrote {OUTPUT_DATA}")
print(f"Wrote {OUTPUT_PDF}")

if __name__ == "__main__":
    main()

```